classification, let $Y \in \{0,1\}^{N\times q}$. We define a graph classification GNN consisting of ℓ message passing layers ($\texttt{MPNN}$), a graph-level readout function ($\texttt{READOUT}$), and classifier head ($\texttt{MLP}$) as follows: $\mathbf{X}^{\ell+1} = \texttt{MPNN}^{\ell+1}\left(\mathbf{X}^{\ell}, \mathbf{A}\right)$, $\mathbf{G} = \texttt{READOUT}\left(\mathbf{X}^{\ell+1}\right)$, and $\hat{Y} = \texttt{MLP}\left(\mathbf{G}\right)$ where $\mathbf{X}^{\ell+1} \in \mathbb{R}^{N\times d_\ell}$ is the intermediate node representation at layer $\ell+1$, $\mathbf{G} \in \mathbb{R}^{1\times d_{\ell+1}}$ is the graph representation, and $\hat{Y} \in \{0,1\}^q$ is the predicted label. When performing node classification, we do not include the $\texttt{READOUT}$ layer, and instead output node-level predictions: $\hat{Y} = \texttt{MLP}\left(\mathbf{X}^{\ell+1}\right)$. We use subscript $_i$ to indicate indexing and $||$ to indicate concatenation.

3.1 Node Feature Anchoring

We begin by introducing a graph anchoring strategy for inducing fully stochastic GNNs. Due to size variability and discreteness, performing a structural residual operation by subtracting two adjacency matrices would be ineffective at inducing an anchored GNN. Indeed, such a transform would introduce artificial edge weights and connectivity artifacts. Likewise, when performing graph classification, we cannot directly anchor over node features, since graphs are different sizes. Taking arbitrary subsets of node features is also inadvisable as node features cannot be considered IID. Further, due to iterative message passing, the network may not be able to converge after aggregating l hops of stochastic node representations (see A.15 for details). Furthermore, there is a risk of exploding stochasticity when anchoring MPNNs. Namely, after l rounds of message passing, a node's representations will have aggregated information from its l hop neighborhood. However, since anchors are unique to individual nodes, these representations are not only stochastic due to their own anchors but also those of their neighbors.

To address both these challenges, we instead fit a d-dimensional Gaussian distribution over the training dataset's input node features which is then used as the anchoring distribution (see Fig. 2). While a simple solution, the fitted distribution allows us to easily sample anchors for arbitrarily sized graphs, and helps manage stochasticity by reducing the complexity of the anchoring distribution, ensuring that overall stochasticity is manageable, even after aggregating the l-hop neighbhorhood. (See A.15 for details.) We emphasize that this distribution is only used for anchoring and does not assume that the dataset's node features are normally distributed. During training, we randomly sample a unique anchor for each node. Mathematically, given anchors $\mathbf{C}^{N\times d} \sim \mathcal{N}(\mu, \sigma)$, we create the anchored node features as: $[\mathbf{X}^0 - \mathbf{C}||\mathbf{X}^0]$. During inference, we sample a fixed set of K anchors and compute residuals for all nodes with respect to the *same* anchor after performing appropriate broadcasting, e.g., $\mathbf{c}^{1\times d} \sim \mathcal{N}(\mu, \sigma)$, where $\mathbf{C} := \texttt{REPEAT}(\mathbf{c}, N)$ and $[\mathbf{X}^0 - \mathbf{C}_k||\mathbf{X}^0]$ is the kth anchored sample. For datasets with categorical node features, anchoring can be performed after embedding the node features into a continuous space. If node features are not available, anchoring can still be performed via positional encodings (Wang et al., 2022b), which are known to improve the expressivity and performance of GNNs (Dwivedi et al., 2022a). Lastly, note that performing node feature anchoring (NFA) is the most analogous extension of Δ-UQ to graphs as it results in fully stochastic GNNs. This is particularly true on node classification tasks where each node can be viewed as an individual sample, similar to a image sample original Δ-UQ formulation.

```
#Node Classification
#(N,inputdim)
X,adj = get_node_dataset()

#Graph Classification
GraphData = get_graph_dataset()
#(Total Num Nodes,inputdim)
X = CONCAT([g.x for g in GraphData],dim=0)

#Create Anchoring Dist
mu = X.mean(dim=0) #(1,inputdim)
std = X.std(dim=0) #(1,inputdim)
#inputdim dimensional Gaussian
AncDist = Normal(mu=mu, std=std)

#Create TRAINING Anchors
#(N,inputdim)
C_Node = AncDist.sample(N)
#(num_nodes(g),inputdim)
C_Graph = AncDist.sample(g.X.shape[0])

#Create INFERENCE Anchors
#(N,inputdim)
C_Node = AncDist.sample(1).repeat(N,dim=0)
#(num_nodes(g),inputdim)
C_Graph =
AncDist.sample(1).repeat(g.X.shape[0],dim=0)

#Anchored Representations
AncNode = CONCAT([X-C_Node, C_Node],dim=1)
AncGraph = CONCAT([X-C_Graph, C_Graph],dim=1)
```

Figure 2: **Node Feature Anchoring Pseudocode.**

3.2 Hidden Layer Anchoring for Graph Classification

While NFA can conceptually be used for graph classification tasks, there are several nuances that may limit its effectiveness. Notably, since each sample (and label) is at a graph-level, NFA not only effectively induces multiple anchors per sample, it also ignores structural information that may be

useful in sampling more *functionally diverse* hypotheses, e.g., hypotheses which capture functional modes that rely upon different high-level semantic, non-linear features. To improve the quality of hypothesis sampling, we introduce hidden layer anchoring below, which incorporates structural information into anchors at the expense of full stochasticity in the network (See Fig. 1):

Hidden Layer and Readout Anchoring: Given a GNN containing ℓ MPNN layers, let $2 \leq r \leq \ell$ be the layer at which we perform anchoring. Then, given the intermediate node representations $\mathbf{X}^{r-1} = \texttt{MPNN}^{r-1}(\mathbf{X}^{r-2}, \mathbf{A})$, we randomly shuffle the node features over the entire batch, ($\mathbf{C} = \text{SHUFFLE}(\mathbf{X}^{r-1}, \text{dim} = 0)$), concatenate the residuals ($[\mathbf{X}^{r-1} - C || C]$), and proceed with the READOUT and MLP layers as usual. (See A.1 for corresponding pseudocode.) Note the gradients of the query sample are not considered when updating parameters, and $\texttt{MPNN}^r$ is modified to accept inputs of dimension $d_r \times 2$ (to take in anchored representations as inputs). At inference, we subtract a single anchor from all node representations using broadcasting. Hidden layer anchoring induces the following GNN: $\mathbf{X}^{r-1} = \texttt{MPNN}^{r-1}(\mathbf{X}^{r-2}, \mathbf{A})$, $\mathbf{X}^{r} = \texttt{MPNN}^{r}\left([\mathbf{X}^{r-1} - \mathbf{C} || \mathbf{C}], \mathbf{A}\right)$, and $\mathbf{X}^{\ell+1} = \texttt{MPNN}^{r+1\ldots\ell}\left(\mathbf{X}^{r}, \mathbf{A}\right)$, and $\hat{Y} = \texttt{MLP}(\texttt{READOUT}\left(\mathbf{X}^{\ell+1}\right))$.

Not only do hidden layer anchors aggregate structural information over r hops, they induce a GNN that is now partially stochastic, as layers $1 \ldots r$ are deterministic. Indeed, by reducing network stochasticity, it is naturally expected that hidden layer anchoring will reduce the diversity of the hypotheses, but by sampling more *functionally diverse* hypotheses through deeper, semantically expressive anchors, it is possible that *naively* maximizing diversity is in fact not required for reliable uncertainty estimation. To validate this hypothesis, we thus propose the final variant, READOUT anchoring for graph classification tasks. While conceptually similar to hidden layer anchoring, here, we simultaneously minimize GNN stochasticity (only the classifier is stochastic) and maximize anchor expressivity (anchors are graph representations pooled after ℓ rounds of message passing). Notably, READOUT anchoring is also compatible with pretrained GNN backbones, as the final MLP layer of a pretrained model is discarded (if necessary), and reinitialized to accommodate query/anchor pairs. Given the frozen MPNN backbone, only the anchored classifier head is trained.

In Sec. 5, we empirically verify the effectiveness of our proposed G-ΔUQ variants and demonstrate that fully stochastic GNNs are, in fact, unnecessary to obtain highly generalizable solutions, meaningful uncertainties and improved calibration on graph classification tasks.

4 Node Classification Experiments: G-ΔUQ Improves Calibration

In this section, we demonstrate that G-ΔUQ improves uncertainty estimation in GNNs, particularly when evaluating *node classifiers* under distribution shifts. To the best of our knowledge, GNN calibration has not been extensively evaluated under this challenging setting, where uncertainty estimates are known to be unreliable (Ovadia et al., 2019). We demonstrate that G-ΔUQ not only directly provides better estimates, but also that combining G-ΔUQ with existing post-hoc calibration methods further improves performance.

Experimental Setup. We use the concept and covariate shifts for WebKB, Cora and CBAS datasets provided by Gui et al. (2022), and follow the recommended hyperparameters for training. In our implementation of node feature anchoring, we use 10 random anchors to obtain predictions with G-ΔUQ. All our results are averaged over 5 seeds and post-hoc calibration methods (described further in App. A.9) are fitted on the in-distribution validation dataset. The expected calibration error and accuracy on the unobserved "OOD test" split are reported.

Results. From Table 1 (and expanded in Table. 12), we observe that across 4 datasets and 2 shifts that G-ΔUQ, *without* any post-hoc calibration (✕), is superior to the vanilla model on nearly every benchmark for better or same accuracy (8/8 benchmarks) and better calibration error (7/8), often with a significant gain in calibration performance. Moreover, we note that combining G-ΔUQ with a particular posthoc calibration method improves performance relative to using the same posthoc method with a vanilla model. Indeed, on WebKB, across 9 posthoc strategies, "G-ΔUQ +<calibration method>" improves or maintains the calibration performance of the corresponding "no G-ΔUQ +<calibration method>" in 7/9 (concept) and 6/9 (covariate) cases. (See App. A.8 for more discussion.) Overall, across post hoc methods and evaluation sets, G-ΔUQ variants are very performant achieving (best accuracy: 8/8), best calibration (6/8) or second best calibration (2/8).

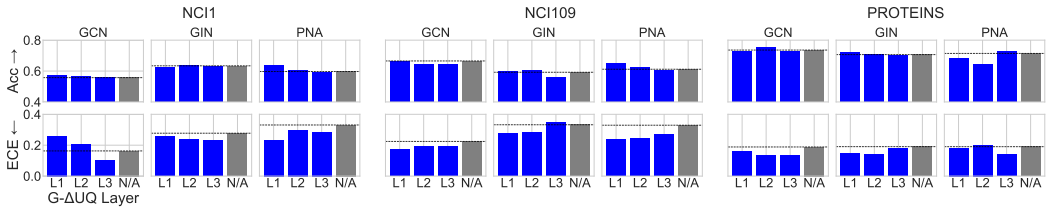

Figure 3: **Effect of Anchoring Layer.** Anchoring at different layers (L1, L2, L3) induces different hypotheses spaces. Variations of stochastic anchoring outperform models without it, and the lightweight READOUT anchoring in particular generally performs well across datasets and architectures.

5 Graph Classification Uncertainty Experiments with G-ΔUQ

While applying G-ΔUQ to node classification tasks was relatively straightforward, performing stochastic centering with graph classification tasks is more nuanced. As discussed in Sec. 3, different anchoring strategies can introduce varying levels of stochasticity, and it is unknown how these strategies affect uncertainty estimate reliability. Therefore, we begin by demonstrating that fully stochastic GNNs are not necessary for producing reliable estimates (Sec. 5.1). We then extensively evaluate the calibration of partially stochastic GNNs on covariate and concept shifts with and without post-hoc calibration strategies (Sec. 5.2), as well as for different UQ tasks (Sec. 5.3). Lastly, we demonstrate that G-ΔUQ's uncertainty estimates remain reliable when used with different architectures and pretrained backbones (Sec. 6).

5.1 Is Full Stochasticity Necessary for G-ΔUQ?

By changing the anchoring strategy and intermediate anchoring layer, we can induce varying levels of stochasticity in the resulting GNNs. As discussed in Sec. 3, we hypothesize that the decreased stochasticity incurred by performing anchoring at deeper network layers will lead to more functionally diverse hypotheses, and consequently more reliable uncertainty estimates. We verify this hypothesis here, by studying the effect of anchoring layer on calibration under graph-size distribution shift. Namely, we find that READOUT anchoring sufficiently balances stochasticity and functional diversity.

Experimental Setup. We study the effect of different anchoring strategies on graph classification calibration under graph-size shift. Following the procedure of (Buffelli et al., 2022; Yehudai et al., 2021), we create a size distribution shift by taking the smallest 50%-quantile of graph size for the training set, and evaluate on the largest 10% quantile. Following (Buffelli et al., 2022), we apply this splitting procedure to NCI1, NCI09, and PROTEINS (Morris et al., 2020), consider 3 GNN backbones (GCN (Kipf & Welling, 2017), GIN (Xu et al., 2019), and PNA (Corso et al., 2020)) and use the same architectures/parameters. (See Appendix A.6 for dataset statistics.) The accuracy and expected calibration error over 10 seeds on the largest-graph test set are reported for models trained with and without stochastic anchoring.

Results. We compare the performance of anchoring at different layers in Fig. 3. While there is no clear winner across datasets and architectures for which *layer* to perform anchoring, we find there is consistent trend across all datasets and architectures the best accuracy and ECE is obtained by a G-ΔUQ variant. Overall, our results clearly indicate that partial stochasticity can yield substantial benefits when estimating uncertainty (though suboptimal layers selections are generally not too harmful). Insofar, as we are the first to focus on partially stochastic anchored GNNs, automatically selecting the anchoring layer is an interesting direction of future work. However, in subsequent experiments, we use READOUT anchoring, unless otherwise noted, as it is faster to train (see App. A.13), and allow our methods to support pretrained models. Indeed, READOUT anchoring (L3) yields top performance for some datasets and architectures such as PNA on PROTEINS, compared to earlier (L1, L2) and, as we discuss below, is very performative on a variety of tasks and shifts.

5.2 Calibration under Concept and Covariate Shifts

Next, we assess the ability of G-ΔUQ to produce well-calibrated models under covariate and concept shift in graph classification tasks. We find that G-ΔUQ not only provides better calibration out of the box, its performance is further improved when combined with post-hoc calibration techniques.